Research Paper Metadata Extractor

Problem Statement

- Create a Python tool to read metadata from PDF files - information like author name, creation date, title, software used etc. that is embedded inside every PDF.
- What is Metadata?
  Metadata = "Data about data". For PDFs, it's hidden info stored in the file:
- PDF contains:
  - Visible content (text/images you see)
  - Metadata (hidden info about the document):
    - Standard metadata*.
    - XMP data.

Functional Requirements (FR)

FR1: Accept .pdf Files Only

- The system shall accept input files strictly in .pdf format.
- The tool shall validate the file extension before processing.
- If the file is not a PDF, the system shall:
  - Display an appropriate error message.
  - Terminate processing for that file.

FR2: Load PDF Using Python Scripts

- The system shall use a Python library such as:
  - pypdf
  - pikepdf
  - pyMuPDF
- The tool shall:
  - Open the metadata in read-only mode.
  - Access document properties without modifying data content.
- Proper exception handling must be implemented.

FR3: Extract Only Standard Metadata

The system shall extract **standard PDF metadata fields only**, not full document text or XMP metadata (unless specified).

Standard Metadata Fields to Extract:

- title, author, subject, keywords, creator, producer, creation_date, modification_date, trapped, page_count

FR4: Convert Extracted Data into JSON Format

- The extracted metadata shall be structured into a JSON object.
- The JSON structure shall follow a predefined schema.

- Example output structure:

```
{
  "title": "Deep Learning for NLP",
  "authors": ["John Doe", "Jane Smith"],
  "subject": "Artificial Intelligence",
  "keywords": ["NLP", "Deep Learning"],
  "producer": "PDFTeX",
  "creation_date": "2024-01-15",
  "modification_date": "2024-01-20",
  "publication_year": 2024
}
```

FR5: Store JSON Output in a File

- The system shall create a .json file for each processed PDF.
- Naming convention example:
  - input.pdf → input_metadata.json
- The system shall:
  - Overwrite existing JSON file (or prompt user).
  - Ensure file writing errors are handled properly.
- The JSON file must:
  - Be readable.
  - Be reusable by other systems.

Non-Functional Requirements (NFR)

NFR1: Handle Multiple User Requests

- The system shall process:
  - Multiple PDFs sequentially.
- The system shall:
  - Maintain stable performance under multiple inputs.
  - Avoid memory leaks.
- It should support:
  - Future scalability for multiprocessing.

NFR2: Do Not Modify PDF File

- The tool shall open PDFs metadata in read-only mode.
- The system shall:
  - Not rewrite metadata.
  - Not alter file content.
- The original file’s metadata must remain unchanged after processing.
- No temporary modifications should persist.

NFR3: CLI-Based Operation

- The system shall operate via Command Line Interface.
- The CLI shall:
  - Display meaningful messages.
  - Provide error handling feedback.
  - Show processing status.
- No GUI or web interface shall be implemented.

NFR4: Performance Efficiency

- The system should:
  - Process a typical research paper (<20MB) within seconds.
- Memory usage should remain minimal.
- The system should not load unnecessary content (like full text).

NFR5: Error Handling

- The system shall log errors appropriately.
- The system shall not crash unexpectedly.

NFR6: Maintainability

- Code must:
  - Follow clean coding standards.
  - Use meaningful variable names.
  - Include documentation/comments.
- Modular structure should allow:
  - Easy addition of XMP extraction in future.

NFR7: Data Integrity

- Extracted metadata must:
  - Match exactly what exists in the PDF.
  - Not be reformatted incorrectly.
- Date formats should be standardized carefully.
- No artificial metadata generation.

Library used :

1. pypdf.
2. pyMupdf.
3. **pikepdf***.
4. **json***.

Research paper Text Extraction

Problem Statement

Convert any PDF (including scanned or handwritten documents) into **searchable, structured JSON text data** by:

1. Converting each PDF page → high-resolution image
2. Running OCR on each image
3. Storing extracted text + metadata in dictionary format
4. Exporting structured results to JSON

Functional Requirements (FR)

FR01: Convert PDF Pages to High-Resolution Images (300 DPI)

The system shall:

- Accept only .pdf files as input.
- Convert each page of the PDF into an image.
- Render images at **minimum 300 DPI** for accurate OCR.
- Support:
  - Multi-page PDFs
  - Large academic/scanned documents
- Libraries can be used:
  - PyMuPDF (fitz).
  - pdf2image.
- Ensure:
  - No modification of the original PDF.
  - Images are temporarily stored or processed in memory.

FR02: Perform OCR on Each Image

The system shall:

- Run OCR on every converted page image.
- Extract:
  - Plain text content
  - Word-level confidence scores
- Support:
  - Printed text
  - Scanned documents
  - Handwritten documents (if possible)
- Allow configurable OCR engine:
  - pytesseract
  - easyocr
  - paddleocr

- Perform optional preprocessing:
  - Grayscale conversion
  - Noise reduction
  - Thresholding
  - Deskewing

FR03: Store Per-Page Data (Structured Format)

For each page, the system shall store:

- page_number
- text
- bounding_boxes
- confidence_scores
- image_dimensions

Bounding box data shall include:

- x
- y
- width
- height

The system shall:

- Store data in a dictionary object internally.
- Preserve page order.
- Handle empty OCR output.

FR04: Generate Structured JSON Output

The system shall generate a JSON file structured as:

```
{
  "filename": "sample.pdf",
  "total_pages": 10,
  "pages": [
    {
      "page_number": 1,
      "page_content": "...",
      "confidence_avg": 91.2
    }
  ]
}
```

The JSON shall:

- Follow consistent schema.
- Use UTF-8 encoding.
- Maintain readable indentation.

- Preserve text formatting where possible.

FR05: Process Directories of PDFs

The system shall:

- Accept:
  - Single PDF file
  - Directory path containing multiple PDFs
- Automatically detect all .pdf files in directory.
- Generate individual JSON output per PDF.
- Continue processing remaining files if one fails.

FR06: Handle Scanned and Handwritten PDFs

The system shall:

- Detect whether PDF contains embedded text or is image-only.
- Apply OCR even if PDF contains selectable text (optional override).
- Attempt handwritten recognition (best-effort basis).
- Allow selection of OCR engine optimized for handwriting (e.g., paddleocr).
- Log warning if OCR confidence is below threshold.

Non-Functional Requirements (NFR)

NFR01: Performance (< 2 Seconds Per Page)

- OCR processing time shall be:
  - Less than 2 seconds per standard A4 page (printed text).
- Image rendering and preprocessing must be optimized.
- Performance measured under:
  - 300 DPI
  - Standard CPU environment

NFR02: Accuracy (> 95% for Printed Text)

- OCR accuracy for clean printed text.
- Confidence scores must be reported.
- Preprocessing should improve recognition quality.
- Lower accuracy expected for:
  - Handwritten text
  - Low-quality scans

NFR03: Memory Usage (< 500MB per PDF)

- System shall:
  - Process pages sequentially to reduce memory load.
  - Avoid storing all page images simultaneously.

- Temporary image buffers should be cleared after processing.
- Large PDFs (100+ pages) must not exceed memory limit.

NFR04: Error Logging
The system shall:
- Log:
  - OCR failures
  - Corrupted PDFs
  - Unsupported formats
  - Low confidence warnings
- Include:
  - Timestamp
  - Filename
  - Error type
- Prevent crashes due to individual page failures.

System Workflow Overview

1. PDF Input
2. PDF → Image Conversion (300 DPI)
3. Image Preprocessing
4. OCR Engine
5. Structured Dictionary
6. JSON Export

Libraries to be used

1. PDF to Image
   - PyMuPDF (fitz).
   - **pdf2image (poppler)*.**

2. **json*** for json encoding.

3. OCR Engines (Not yet fixed)
   - Tesseract.
   - easyocr.
   - paddleocr.

Podcast MetaData Extractor

Problem Statement

- Create a metadata extraction system for MP3 (podcast-focused) audio files that:
  - Reads ID3 tags (v1 + v2.x)
  - Extracts MPEG frame-level technical properties
  - Handles large podcast collections
  - Exports structured **JSON / CSV**
  - Works reliably across OS platforms
- Podcast MP3 files typically contain:
  - Stereo or Mono audio
  - Variable bitrate (VBR)
  - Long duration (30 min – 3 hrs)
  - Rich ID3 tags
  - Embedded cover art
  - Chapter markers (sometimes)

Functional Requirements

FR01: Extract ID3v1 / ID3v2 Tags

- Extract human-readable metadata stored in ID3 tags.
- MP3 files may contain:
- From ID3 Frames:

Logical Field	ID3 Frame ID
Title	TIT2
Artist	TPE 1
Album	TALB
Genre	TCON
Track	TRCK
Year	TYER / TDRC
Comment	COMM
Album Artist	TPE 2
Podcast Episode	TIT3
Publisher	TPUB

Podcast-Specific Metadata

- Podcast Name (Album)
- Episode Title (Title)
- Episode Number (Track)
- Host(s) (Artist)
- Description (Comment)
- Cover Art (APIC frame)

Handling Strategy

- Prefer ID3v2 over ID3v1 if both exist
- Normalize encoding to UTF-8
- Trim null bytes and whitespace
- Return null for missing fields (not error)

FR02: Extract Audio Properties (Technical MPEG Metadata)

- Extract MPEG frame-level technical audio information.
- Audio Fields to Extract : duration_seconds, bitrate_kbps, sample_rate_hz, channels, channel_mode, mpeg_version, file_size_bytes etc.
- For podcast cataloging we should detect:
  - Is it stereo? (channels == 2)
  - Is bitrate low? (e.g., 64kbps mono typical for speech)
  - Long-form content? (duration > 1800 sec)
  - Compression efficiency
  - Spoken word optimization
- We can derive:
  - Estimated file quality
  - Bandwidth category
  - Distribution readiness

FR03: Processing

- Filter .mp3 only
- No modifying audio file metadata.
- Logs on error (any failures)
- Command line interface.

FR04: Output JSON / CSV

- Structured Output json output.
- JSON Schema

Example:

{ "filename": "episode1.mp3",
"title": "AI Revolution",
"album": "TechTalk Podcast",
"duration_seconds": 3600,
"bitrate_kbps": 128,
"sample_rate_hz": 44100,
"channels": 2,
"channel_mode": "stereo",
"file_size_bytes": 58239482, …… }

Non-Functional Requirements

NFR01: Metadata extraction

Technical Interpretation:

- Metadata extraction only (no full audio decoding)
- Avoid waveform loading
- Avoid reading entire file into memory
- Parse headers only

NFR02: Python 3.9+

Avoid:

- OS-specific syscalls
- External dependencies requiring compilation

Prefer:

- Pure Python libraries
- Standard library modules

NFR03: Memory Usage

Strategies:

- No full file buffering
- No audio waveform storage
- Release objects after processing
- Process file-by-file

For large batch:

- Use generator-based iteration
- Avoid accumulating entire dataset before writing

Architecture Flow

1. File Input
2. MP3 Header Parsing
3. ID3 Frame Extraction
4. Schema Validation
5. JSON/CSV Writer

Libraries Under Feasibility Evaluation

The following Python libraries are currently being evaluated for metadata extraction.
At this stage, no library has been finalized. A comparative feasibility analysis is in progress.

- tinytag
- mutagen
- music-tag
- eyed3

Podcast Transcription Extraction

Problem Statement

- Design and implement a system that processes MP3 podcast audio files, extracts technical metadata, performs speech-to-text transcription, and generates structured JSON output.
- For multi-channel podcast recordings (e.g., stereo where different speakers are recorded on separate channels), the system must perform channel-wise transcription and generate separate textual outputs per channel.
- The solution must support batch processing, long-duration podcast files, and scalable JSON output formatting.

Functional Requirements (Updated & Expanded)

FR01: MP3 Input Validation & Channel Detection

The system must:

- Accept .mp3 files (single or batch)
- Validate file integrity
- Extract technical audio properties before transcription.
- Must detect fields like : channels, channel_mode, sample_rate_hz, bitrate_kbps, duration_seconds.
- Channel Logic : Mono(Single transcription), stereo(Split channels and transcribe independently).

FR02: Channel-Wise Audio Separation

- If the podcast contains multiple channels:
  - Extract left and right channels separately
  - Preserve original sampling rate
  - Convert to mono streams per channel
- Engineering Note:
  - No mixing before transcription
  - Avoid lossy re-encoding
  - Maintain synchronization timestamps
- This enables:
  - Cleaner speaker separation (if recorded independently)
  - Reduced cross-talk errors

FR03: Speech-to-Text Transcription

- Mono File:
  - Full transcription as single text body
- Multi-Channel File:
  - Independent transcription per channel
  - Output labeled as: channel_1, channel_2.

FR04: JSON Output Structure

Mono Example:

```
{
  "filename": "episode1.mp3",
  "channels": 1,
  "duration_seconds": 3600,
  "transcription": "Full episode transcription text..."
}
```

Multi-Channel Example:

```
{
  "filename": "episode2.mp3",
  "channels": 2,
  "duration_seconds": 4200,
  "transcriptions": {
    "channel_1": "Host speech text...",
    "channel_2": "Guest speech text..."
  }
}
```

Non-Functional Requirements (Updated)

NFR1: Performance

- Channel splitting overhead minimal
- Efficient chunk processing
- Avoid re-encoding overhead

NFR2: Memory Efficiency (for future enhancements)

- Do NOT load full 2-hour audio into RAM
- Use stream or chunk-based processing()
- Release channel buffers after transcription

NFR3: Accuracy

- Channel separation must not degrade clarity
- ≥ 90% transcription accuracy for clean speech
- Avoid cross-channel leakage

NFR4: Data Handling

- No permanent storage of audio and Temporary channel files auto-deleted
- JSON output validated before writing

Libraries Under Feasibility Evaluation (Still Not Finalized)

For Audio Processing:

- pydub
- librosa
- ffmpeg-python
- native wave handling

For Transcription:

- Whisper
- Faster-Whisper
- Vosk
- Cloud APIs

Library decision pending:

- Accuracy benchmarking
- Memory profiling
- Long podcast stress testing
- Channel separation validation